

ONLINE FOOD ORDERING SYSTEM

Project Documentation Report

Submitted By: Ennuhang Rai

Course: Bachelor In Information Technology.

Technology Used: Python , SQLite, HTML, CSS.

Academic Year: 2026

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Objectives
4. Scope of the Project
5. Technology Stack
6. System Architecture
7. Database Design
8. Users Table
9. Stores Table
10. Products Table
11. Orders Table
12. Transaction Record Table
13. Entity Relationship Overview
14. User Module
15. Store Registration Module
16. Product Management Module
17. Search Module
18. Order Processing Module
19. Wallet Management
20. File Upload Feature

21. Authentication and Security

22. Testing and Validation

23. Sample Test Case

24. Challenges Faced

25. Solutions Implemented

26. Advantages

27. Limitations

28. Future Enhancements

Abstract

The Online Food Ordering System is a web-based application developed using Flask (Python) as the backend framework and SQLite as the database system. The main purpose of this project is to provide a digital platform where customers can easily search for restaurants, view available food items, and place orders online. At the same time, store owners can manage their business through the system.

Customers are able to create accounts, log in securely, search for shops, filter results, and select food items from the menu. The system calculates the total price, including delivery charges if applicable, and updates the wallet balance after order confirmation. Customers can also view their previous orders and transaction history, making the system organized and user-friendly.

Store owners can register their stores, add food products, upload images, set prices, and manage available quantities. They can also view incoming orders, update order status, and monitor transactions. The system automatically updates inventory levels and records financial transactions to maintain accuracy.

From a technical perspective, this project demonstrates important concepts such as database management, session handling, authentication, relational data structure, and file upload functionality. It provides practical experience in backend development and shows how real-world food delivery platforms operate on a smaller scale.

Introduction

The Online Food Ordering System is designed to simulate the functionality of real-world food delivery platforms such as Foodmandu or Uber Eats. The main objective of this system is to create a centralized web application where customers and store owners can interact efficiently. It provides a digital solution for managing food orders, business operations, and transactions within a single platform.

Through this system, customers can register, log in, search for restaurants, view menus, and place orders online. Store owners, on the other hand, can register their stores, add food products, manage inventory, and process incoming orders. This interaction between customers and store owners is handled through a structured and secure backend system.

The project mainly focuses on implementing key web development concepts such as user authentication, session management, relational database design, and product management. It demonstrates how different database tables (Users, Stores, Products, Orders, and Transactions) are connected using foreign key relationships to maintain data consistency.

Overall, this project provides practical experience in building a real-world web application and helps in understanding how online food delivery platforms operate in terms of backend logic, database management, and user interaction.

Objectives

The primary objective of the Online Food Ordering System is to develop a dynamic and interactive web-based platform that allows users to order food online efficiently. The system is designed to simulate real-world food delivery applications while focusing on backend logic and database management.

One of the main goals is to implement secure user authentication and session management. This ensures that only authorized users can access specific features such as placing orders, managing stores, or viewing transaction history. Proper session handling also helps maintain user activity throughout the system.

Another important objective is to enable store registration and product management. Store owners should be able to register their businesses, add food items, upload images, update prices and quantities, and manage their inventory easily through the platform.

The system also aims to implement proper order processing and transaction tracking. Whenever an order is placed, completed, or canceled, the database should update automatically, and transaction records should be generated to maintain financial accuracy.

Finally, this project demonstrates the implementation of relational database concepts, including primary keys, foreign keys, table relationships, and data consistency. It provides practical experience in designing and managing structured databases within a real-world application scenario.

Scope of the Project

The scope of the Online Food Ordering System includes the development of a complete web-based platform that manages food ordering and business operations in a structured manner. The system supports user registration and secure login, allowing customers and store owners to access features based on their roles.

Within the platform, store owners can create and manage their stores by adding product details such as meal name, price, quantity, and images. They can update or remove products and monitor incoming orders. Customers can search for available stores, browse menus, select food items, and place orders directly through the website.

The system also includes wallet management functionality, where users can add balance and use it to complete transactions. Every financial activity, such as payments, refunds, and recharges, is recorded in the transaction history to ensure transparency and accuracy.

This project is mainly suitable for small-scale restaurant management systems or academic demonstrations. While it simulates real-world food delivery platforms, it is designed for local deployment and learning purposes rather than large-scale commercial use.

Technology Stack

The Online Food Ordering System is developed using a combination of backend, frontend, and database technologies that work together to create a complete web application.

The backend of the system is built using Python and the Flask framework. Flask is a lightweight web framework that allows developers to handle routing, request processing, session management, and server-side logic efficiently. It is simple yet powerful, making it suitable for academic projects and small-scale applications.

For database management, the system uses SQLite (HWDB.db). SQLite is a lightweight, serverless relational database that stores data in a single file. It supports structured queries, foreign key relationships, and transaction handling, making it ideal for managing users, stores, products, orders, and transaction records within the application.

The frontend of the system is developed using HTML and CSS. HTML is used to structure the web pages, forms, tables, and navigation elements, while CSS is used to style the interface and improve user experience.

To connect the Flask backend with the SQLite database, the project uses Flask-SQLAlchemy, which is an Object Relational Mapping (ORM) tool. ORM allows developers to interact with the database using Python objects instead of writing raw SQL queries, making database operations cleaner, safer, and easier to manage.

System Architecture

The Online Food Ordering System follows a three-layer architecture, which helps organize the application into separate components based on their responsibilities. This structured approach improves maintainability, scalability, and clarity of the system design.

The first layer is the Presentation Layer, which consists of HTML templates and CSS styling. This layer is responsible for displaying the user interface, including forms, navigation menus, product listings, and order details. It interacts with users by collecting input and displaying output generated by the backend.

The second layer is the Application Layer, which is implemented using Flask routes and backend logic. This layer handles user requests, processes data, performs validation, manages sessions, and communicates with the database. It acts as the bridge between the frontend and the database. For example, when a user places an order, the application layer verifies the input, calculates the total price, updates inventory, and stores transaction records.

The third layer is the Data Layer, which uses SQLite as the database system. This layer is responsible for storing and retrieving structured data such as user accounts, store details, product information, orders, and transaction records. The database ensures data consistency and maintains relationships between different tables using primary and foreign keys.

By separating the system into these three layers, the application becomes more organized and easier to manage. Each layer performs a specific role, ensuring a clean and efficient design.

Database Design

The Online Food Ordering System uses a relational database design to store and manage all application data in an organized and structured manner. The database is implemented using SQLite and consists of multiple interconnected tables, including Users, Stores, Products, Orders, Transaction_Record, and Process_Order.

Each table serves a specific purpose within the system. The Users table stores account information such as user ID, username, encrypted password, wallet balance, and personal details. The Stores table contains information about registered shops, including store name, location, category, and owner reference. The Products table stores details about food items, such as product name, price, quantity, image data, and store association.

The Orders table records customer orders, including order status, timestamps, and total price. The Process_Order table helps manage the relationship between orders and the products included in each order. The Transaction_Record table tracks all financial activities such as payments, refunds, and wallet recharges to ensure transparency and accountability.

```
$ sqlite3 HWDB.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
Orders          Products        Transaction_Record
Process_Order   Stores          Users
sqlite> 
```

Users Table

The Users table is one of the most important tables in the Online Food Ordering System. It stores all account-related information for registered users, including both customers and store owners. Each user is uniquely identified using a UID (User ID), which serves as the primary key of the table.

The table includes fields such as username and hashed password. Passwords are not stored in plain text; instead, they are encrypted (hashed) before being saved in the database to ensure security. This prevents unauthorized access even if the database is exposed.

```
sqlite> SELECT * FROM Users;  
1|arjun123|c23a395448ca2b2570ed2acac1d1261a421487b6d6490de05ad840962d9f6ec1|Arjun Thapa|0|27.7172|27.7172|9841234567|0  
2|sujan123|eecc73aa115ab72a34f062d6833cb68a3bc99aac7945bf48697243ae084b1697|Sujan Karki|1|27.7172|85.324|9845678910|1000  
sqlite> 
```

Stores Table

The Stores table is responsible for storing all information related to registered shops within the Online Food Ordering System. Each store is uniquely identified by a Store ID (SID), which serves as the primary key of the table. This ensures that every store in the system can be distinguished from others.

The table contains important details such as the store name, latitude, longitude, phone number, and food category. The latitude and longitude values are used to calculate delivery distance and determine delivery charges. The food category helps customers filter and search for specific types of restaurants, such as Nepali, Chinese, or Fast Food.

One of the most important fields in this table is the owner ID (S_owner). This field acts as a foreign key that references the Users table. It establishes a relationship between the store and the user who owns it. This means that each store is linked to a specific registered user account. If a user has not registered a store, they will not have any records in the Stores table.

```
sqlite> SELECT * FROM Stores;  
2|HimalayanCafe|27.7172|85.324|9845678910|Nepali|2  
sqlite> 
```

Products Table

The Products table stores all information related to food items available in each registered store. Each product is uniquely identified by a Product ID (PID), which serves as the primary key of the table. This ensures that every meal item can be individually managed and tracked within the system.

The table includes important details such as the product name, price, and quantity. The price field defines the cost of the meal, while the quantity field represents the available stock. Whenever a customer places an order, the quantity is automatically reduced to reflect the updated inventory level. If an order is canceled, the quantity is restored accordingly.

```
rice": 150, "P_store": 2}], "Subtotal": 300}|2
sqlite> SELECT PID, P_name, P_price, P_quantity FROM Products;
1|Momo|150|50
2|Chowmein|200|40
3|Burger|150|48
4|momo|200|4
5|coffee|275|50
sqlite> 
```

Orders Table

The Orders table is responsible for storing all information related to customer orders placed within the system. Each order is uniquely identified using an Order ID (OID), which serves as the primary key of the table. This allows the system to track every order individually.

The table contains important details such as the customer ID, store ID, order creation time, and the current order status. The order status indicates whether the order is *Not Finished*, *Finished*, or *Canceled*. This helps both customers and store owners monitor the progress of each order.

In addition to basic order information, the system also stores product-related details such as ordered items and their quantities. When an order is placed, the system updates the product inventory and deducts the appropriate amount from the customer's wallet. If an order is canceled, the product quantity is restored, and the customer receives a refund.

[illegible]

Transaction Record Table

The Transaction Record table is responsible for tracking all financial activities within the Online Food Ordering System. It records every change in a user’s wallet balance, ensuring transparency and proper financial management. Each transaction is uniquely identified by a Transaction ID, which serves as the primary key of the table.

This table stores details such as the transaction type (Payment, Receive, or Recharge), the amount changed, the date and time, and the trader involved in the transaction. For example, when a customer places an order, the wallet balance decreases, and a payment record is created. At the same time, the store receives the payment, and a corresponding receive record is generated.

Similarly, when a user recharges their wallet, a recharge transaction is recorded. If an order is canceled, refund transactions are also created for both the customer and the store.

HomeShopsMy OrdersShop OrdersTransaction Records

Transaction Records

All

Record ID	Action	Time	Trader	Amount
1	Recharge	2026/02/18 15:52	Sujan Karki	1000
2	Payment	2026/02/18 15:53	HimalayanCafe	-310
3	Recieve	2026/02/18 15:53	Sujan Karki	310
4	Payment	2026/02/18 16:08	HimalayanCafe	-310
5	Recieve	2026/02/18 16:08	Sujan Karki	310
6	Recieve	2026/02/18 16:25	HimalayanCafe	310
7	Payment	2026/02/18 16:25	Sujan Karki	-310

Entity Relationship Overview

The Online Food Ordering System follows a structured relational database design where different tables are connected through defined relationships. The main relationship flow in the system can be represented as:

Users → Stores → Products → Orders → Transactions

The Users table is the foundation of the system. Every registered user can either act as a customer or a store owner. When a user registers a business, a corresponding record is created in the Stores table, linking the store to its owner through a foreign key relationship.

Each store can have multiple food items listed in the Products table. These products are connected to the store using a store reference ID. This ensures that every product belongs to a specific store and cannot exist independently.

When a customer places an order, the information is recorded in the Orders table. Orders are linked to both the user (customer) and the store, creating a clear transactional relationship between the two parties.

Finally, any financial activity related to the order—such as payments, refunds, or wallet recharges—is recorded in the Transactions table. This ensures that all monetary changes are properly tracked and stored.

User Module

The User Module is responsible for managing all functionalities related to user accounts within the Online Food Ordering System. It handles essential operations such as registration, login authentication, profile management, wallet top-up, and viewing order history.

During registration, users create an account by providing necessary information such as username, password, name, and contact details. The system securely encrypts (hashes) the password before storing it in the database to ensure data security. Once registered, users can log in using their credentials, and session management is used to maintain their authenticated state while navigating the website.

The module also allows users to manage their profiles and update relevant information if needed. One of the key features of this module is the wallet top-up system, which enables users to add funds to their account. These funds are later used to place orders. Every wallet recharge or payment transaction is automatically recorded in the Transaction Record table.

Home

Shops

My Orders

Shop Orders

Transaction Records

My Orders

All

Order ID	Status	Start Time	End Time	Shop Name	Total Price	Order Details	Action	Select
1	Canceled	2026/02/18 15:53		HimalayanCafe	310	<button>Order Details</button>		
2	Not finished	2026/02/18 16:08		HimalayanCafe	310	<button>Order Details</button>	<button>Cancel Order</button>	☐

Cancel Seleted Orders

Store Registration Module

The Store Registration Module allows users to register their business within the Online Food Ordering System. This feature is specifically designed for users who want to operate as store owners and manage their own food shop on the platform.

To register a store, the owner must provide essential details such as the store name, food category, latitude, and longitude. The category helps customers search and filter stores based on cuisine type, while the location details are used to calculate delivery distance and delivery fees.

[Home](#) [Shops](#) [My Orders](#) [Shop Orders](#) [Transaction Records](#)

Start a business

shop name	shop category	latitude	longitude
HimalayanCafe	Nepali	27.7172	85.324

ADD

meal name	
price	quantity

上傳圖片

No file chosen

Product Management Module

The Product Management Module allows store owners to manage all food items available in their shop. This module provides full control over the store's menu and inventory within the Online Food Ordering System.

Store owners can add new meals by entering details such as product name, price, and available quantity. They can also upload images of the food items to make the menu more visually appealing and informative for customers. The image upload functionality enhances user experience and helps customers make better purchasing decisions.

In addition to adding products, store owners can update existing items, including modifying prices or adjusting available stock. When an order is placed, the system automatically reduces the product quantity. If an order is canceled, the inventory is restored accordingly, ensuring accurate stock management.

ADD

meal name

price

quantity

上傳圖片

Choose Files No file chosen

Add

No file chosen

#	Picture	meal name	price	Quantity	Edit	Delete
1		Burger	150	46	Edit	Delete
2		momo	200	4	Edit	Delete
3		coffee	275	50	Edit	Delete

Search Module

The Search Module enables customers to find restaurants and food items quickly and efficiently within the Online Food Ordering System. This module plays an important role in improving user experience by allowing flexible filtering and sorting options.

Customers can search for stores based on different criteria such as store name, food category, price range, and meal name. For example, a user can search for a specific restaurant by entering its name, or filter restaurants that serve a particular type of cuisine. The price range filter helps customers find meals within their budget, while the meal name filter allows them to search for specific food items.

In addition to filtering, the module also provides sorting options, such as sorting results by price or other attributes. This helps users organize search results according to their preferences.

[Home](#) [Shops](#) [My Orders](#) [Shop Orders](#) [Transaction Records](#)

Profile

Account:sujan123, Name:Sujan Karki, manager, PhoneNumber: 9845678910, location: 27.7172, 85.324 [Edit location](#) walletbalance: 1000 [Top Up](#)

Search

Shop

distance

Price ~

Meal

category

ordering

desc ☐

order

[Search](#)

#	shop name	shop category	Distance
---	-----------	---------------	----------

[pre](#) [1](#) [nxt](#)

Order Processing Module

The Order Processing Module is one of the core components of the Online Food Ordering System. It manages the entire workflow that occurs when a customer places an order.

When a customer selects food items and confirms the order, the system first validates the input. It checks whether the requested quantity is available in stock and whether the customer has sufficient wallet balance. If all conditions are satisfied, the system proceeds with order processing.

Once the order is successfully placed, the product quantity is automatically reduced from the inventory to reflect the updated stock level. At the same time, the customer's wallet balance is deducted by the total order amount, which may include delivery charges if applicable. The store's balance is increased accordingly.

Home

Shops

My Orders

Shop Orders

Transaction Records

My Orders

All

Order ID	Status	Start Time	End Time	Shop Name	Total Price	Order Details	Action	Select
1	Canceled	2026/02/18 15:53		HimalayanCafe	310	<button>Order Details</button>		
2	Not finished	2026/02/18 16:08		HimalayanCafe	310	<button>Order Details</button>	<button>Cancel Order</button>	☐

Cancel Selected Orders

Wallet Management

The Wallet Management feature allows customers to manage their account balance within the Online Food Ordering System. Instead of using an external payment gateway, the system uses a digital wallet model to simulate online transactions.

Customers can top up their wallet balance by entering a positive amount in the recharge section. Once the recharge request is validated, the system increases the user's wallet balance accordingly. To maintain transparency and accountability, every recharge action generates a corresponding entry in the Transaction Record table.

When a customer places an order, the total amount is deducted directly from the wallet balance. Similarly, if an order is canceled, the deducted amount is refunded back to the customer's wallet. Each of these actions also creates transaction records to ensure proper financial tracking.

Home	Shops	My Orders	Shop Orders	Transaction Records
Transacrion Records				
All ▾				
Record ID	Action	Time	Trader	Amount
1	Recharge	2026/02/18 15:52	Sujan Karki	1000
2	Payment	2026/02/18 15:53	HimalayanCafe	-310
3	Recieve	2026/02/18 15:53	Sujan Karki	310
4	Payment	2026/02/18 16:08	HimalayanCafe	-310
5	Recieve	2026/02/18 16:08	Sujan Karki	310
6	Recieve	2026/02/18 16:25	HimalayanCafe	310
7	Payment	2026/02/18 16:25	Sujan Karki	-310

File Upload Feature

The Online Food Ordering System includes a File Upload Feature that allows store owners to upload images of their food products. This functionality improves the visual presentation of the menu and enhances the overall user experience.

The system uses multipart/form-data encoding in HTML forms to handle file uploads. This encoding type is necessary when submitting files such as images through a web form. Without it, the browser would not be able to send image data correctly to the server.

When a store owner uploads an image, the backend processes the file and stores it as BLOB (Binary Large Object) data in the database. Along with the image data, the system also stores the image type (for example, image/jpeg or image/png) to ensure proper rendering when displaying the image on the webpage.

Whenever customers view the menu, the system retrieves the image from the database, converts it into a displayable format, and shows it alongside the product details. This integration between file handling and database storage demonstrates backend file processing and data management capabilities.

上傳圖片

Choose Files

No file chosen

Add

#	Picture	meal name	price	Quantity	Edit	Delete
---	---------	-----------	-------	----------	------	--------

Authentication and Security

Authentication and security are essential components of the Online Food Ordering System. The system ensures that user data is protected and that only authorized individuals can access specific features.

One of the primary security measures implemented in this project is password hashing. Instead of storing passwords in plain text, the system encrypts (hashes) them using a secure hashing algorithm before saving them in the database. This means that even if someone gains access to the database, they cannot directly read users' original passwords. Hashing significantly reduces the risk of unauthorized account access.

```
Process_order.py Stores Users
sqlite> SELECT * FROM Users;
1|arjun123|c23a395448ca2b2570ed2acac1d1261a421487b6d6490de05ad840962d9f6ec1|Arjun Thapa|0|27.7172|27.7172|9841234567|0
2|sujan123|eccc73aa115ab72a34f062d6833cb68a3bc99aac7945bf48697243ae084b1697|Sujan Karki|1|27.7172|85.324|9845678910|1000
sqlite> 
```

Testing and Validation

Testing and validation were conducted to ensure that all modules of the Online Food Ordering System function correctly and meet the project requirements. Each feature was carefully tested to verify accuracy, reliability, and proper database updates.

The registration and login modules were tested to confirm that users can create accounts successfully, passwords are securely stored, and session management works properly. Invalid login attempts and incorrect inputs were also tested to ensure appropriate error messages are displayed.

The store creation module was tested to verify that store details are correctly inserted into the database and properly linked to the owner through foreign key relationships. Similarly, the product insertion module was tested to confirm that meals, prices, quantities, and images are correctly stored and displayed.

Sample Test Case

To verify the correct functionality of the Product Management Module, a test case was performed for adding a new product to the system.

Test Case: Add Product

Input:

- Product Name: Momo
- Price: 200
- Quantity: 50

Result:

The test was successful. The product was stored correctly in the database and displayed properly on the store page. The data could also be verified directly in the Products table, confirming accurate insertion and functionality.

上傳圖片

Choose Files No file chosen

Add

#	Picture	meal name	price	Quantity	Edit	Delete
1		Burger	150	46	Edit	Delete
2		momo	200	4	Edit	Delete
3		coffee	275	50	Edit	Delete

Challenges Faced

During the development of the Online Food Ordering System, several technical challenges were encountered and carefully resolved. One of the main issues involved database mismatches, particularly when connecting the Flask application to the correct SQLite database file. In some cases, the application was pointing to a different database file, which resulted in missing tables or empty query results. This issue was resolved by correctly configuring the database URI and ensuring that the schema was properly initialized.

Another significant challenge was related to session management. There were instances where the system did not correctly recognize the logged-in user, causing errors such as “Please register your store first” even when the store existed in the database. This occurred because session variables were not properly synchronized with database records. The issue was fixed by ensuring that user IDs were correctly stored in sessions after login and properly referenced during store and order operations.

Solutions Implemented

To address the challenges encountered during development, several corrective measures were implemented to ensure system stability and proper functionality.

First, the database configuration was carefully reviewed and corrected. The application was configured to connect to the appropriate SQLite database file, and the database schema was properly initialized. This resolved issues related to missing tables and incorrect data retrieval.

Second, proper owner ID mapping was ensured to maintain accurate relationships between Users and Stores tables. Foreign key references were verified and updated so that each store was correctly linked to its respective owner. This eliminated errors related to store recognition and access control.

Advantages

The Online Food Ordering System offers several advantages in terms of design, functionality, and implementation.

One of the main advantages is that it is a lightweight and efficient system. Since it uses Flask and SQLite, the application runs smoothly without requiring complex server configurations. SQLite operates as a serverless database, which reduces system overhead and simplifies management.

Another advantage is that the system is easy to deploy. It can be hosted locally using a simple setup without the need for advanced infrastructure. This makes it suitable for academic projects, demonstrations, and small-scale applications.

The project also demonstrates real-world database relationship implementation. It effectively uses primary keys and foreign keys to connect tables such as Users, Stores, Products, Orders, and Transactions. This structured relational design ensures data consistency and reflects how professional database systems operate.

Limitations

Although the Online Food Ordering System successfully implements core functionalities, it has certain limitations that restrict its scalability and real-world application.

One of the main limitations is the absence of an integrated payment gateway. The system uses a wallet simulation instead of connecting to real payment services such as online banking, credit cards, or digital payment platforms. While this is sufficient for academic purposes, it does not support real financial transactions.

Another limitation is the lack of a delivery tracking feature. The system calculates delivery fees based on distance, but it does not provide real-time tracking of delivery status, rider location, or estimated delivery time. In real-world food delivery platforms, tracking is an essential feature.

Additionally, the system has a basic user interface design. Although it is functional and user-friendly, it does not include advanced UI/UX elements, animations, or responsive design optimization for different devices.

These limitations highlight areas where the system can be further enhanced to meet commercial standards and provide a more complete user experience.

Future Enhancements

The Online Food Ordering System can be further improved by adding several advanced features to enhance functionality, security, and user experience.

One major improvement would be the integration of a payment gateway. Instead of using a simulated wallet system, the platform could support real online payment methods such as credit cards, debit cards, or digital payment services. This would make the system more realistic and suitable for commercial use.

Another useful enhancement is the development of an admin dashboard. An admin panel would allow system administrators to monitor users, stores, transactions, and overall platform activity. It would also help manage reports, detect suspicious activity, and maintain system control.

The addition of a rating and review system would improve user engagement. Customers could provide feedback on stores and food items, helping other users make informed decisions and encouraging stores to maintain quality service.

Implementing email notifications would enhance communication. The system could send confirmation emails for registration, order placement, order cancellation, and transaction updates.

Conclusion

The Online Food Ordering System successfully demonstrates a full-stack web application using Flask and SQLite. It integrates authentication, relational database design, image handling, and transaction management. The system provides practical experience in backend development and database-driven application design.